



LICEO BICENTENARIO MARIO BERTERO CEVASCO

PROGRAMACIÓN PYTHON — 4TO MEDIO 2026

PROFESOR: DIEGO VARGAS PALOMINOS

GUÍA DE ESTUDIO

CLASE 24B — FUNCIONES

`return` · guardar en variable · reutilizar

Objetivo de la clase: Usar funciones con `return`, guardando siempre su resultado en una variable antes de utilizarlo en un cálculo o mensaje posterior, con método.

Propósito: *El método es seguir siempre el mismo paso, sin importar el caso, para no improvisar a mitad de camino. Hoy lo practicamos guardando siempre en una variable lo que devuelve una función, antes de usarlo.*

Nombre: _____

Fecha: _____

Términos que repasamos hoy

No son conceptos nuevos — son la base que hoy consolidamos

- **Parámetro / argumento:** el parámetro es la variable que la función espera recibir (se define en `def nombre(parametro):`); el argumento es el valor real que le entregas al llamarla.
- **return:** entrega un valor de vuelta a quien llamó la función, para que pueda guardarse en una variable y usarse después.
- **None:** lo que queda guardado cuando una función no tiene **return**, o cuando usa `print()` en vez de devolver el valor.
- **Patrón "guardar antes de usar":** escribir siempre `variable = funcion(argumento)` — nunca usar el resultado de una función sin haberlo guardado primero.

El problema: resultados que se pierden o que nunca existieron

Cuando un programa usa funciones, hay dos formas típicas de que algo salga mal aunque el código "se vea bien". La primera: llamas la función, ves el resultado aparecer en pantalla, pero cuando intentas usarlo un poco más abajo el programa se detiene con un error que dice que esa variable no existe — porque en realidad nunca la guardaste, solo la imprimiste de paso. La segunda: guardas el resultado en una variable, pero cuando la usas después descubres que su valor es `None` — porque la función mostraba algo en pantalla con `print()`, pero nunca lo entregó de vuelta con **return**.

Fíjate en este programa:

```
def saluda(nombre):  
    print("Hola,", nombre)  
  
valor = saluda("Camila")  
print(valor)
```

```
>> Hola, Camila  
>> None
```

La función `saluda` sí hace algo — imprime el saludo — pero no tiene **return**, así que lo que queda guardado en `valor` es `None`. Que una función "muestre algo en pantalla" no significa que ese algo quede disponible para usarlo después: solo **return** lo deja disponible. Ambos problemas de arriba se evitan con la misma regla, que hoy convertimos en un hábito fijo: definir, consultar los datos, evaluar guardando el resultado, y recién ahí usar ese resultado.

El patrón general — guardar antes de usar. Existe una regla que ordena cualquier programa que use una función, en 4 partes: primero **definir** la función, después **consultar** los datos que necesita, luego **evaluarla** guardando su resultado en una variable (`variable = funcion(argumento)`), y recién ahí **imprimir** o usar ese resultado.

```
# DEFINICION
def ganancia_venta(precio_venta, costo_producto):
    return precio_venta - costo_producto

# CONSULTA
precio_ingresado = 1200
costo_ingresado = 700

# EVALUACION
ganancia_helado = ganancia_venta(precio_ingresado, costo_ingresado)

# IMPRESION
print("Ganancia de esa venta:", ganancia_helado)
```

Idea clave: esta regla general — definición, consulta, evaluación, impresión — ordena cualquier programa que use una función, y evita el error de usar un resultado antes de haberlo guardado.

Reutilizar el valor guardado. Una vez guardado en una variable, ese resultado puede usarse como parte de un cálculo posterior — no se "pierde" después de imprimirse.

```
ganancia_bebida = ganancia_venta(900, 400)
ganancia_total = ganancia_helado + ganancia_bebida
print("Ganancia total:", ganancia_total)
```

Idea clave: encadenar cálculos usando variables guardadas es la base de programas más largos — cada resultado se convierte en el insumo del siguiente paso.

Resolvamos la Feria de Emprendimiento — paso a paso

La Feria de Emprendimiento del liceo tiene puestos que venden distintos productos. Cada puesto quiere calcular la ganancia de una venta (precio de venta menos costo del producto), y después de dos ventas del día, saber si alcanzó la meta de recaudación que se fijó la Feria.

Paso 1 — Definimos la función que calcula la ganancia de una venta.

```
def ganancia_venta(precio_venta, costo_producto):
    return precio_venta - costo_producto
```

Definir la función todavía no calcula nada — solo queda lista para usarse con cualquier venta del día.

Paso 2 — Pedimos los datos de la primera venta y guardamos su ganancia.

```
precio_1 = int(input("¿Precio de venta del primer producto? "))
costo_1 = int(input("¿Costo del primer producto? "))
ganancia_1 = ganancia_venta(precio_1, costo_1)
```

Paso 3 — Repetimos para la segunda venta, en su propia variable.

```
precio_2 = int(input("¿Precio de venta del segundo producto? "))
costo_2 = int(input("¿Costo del segundo producto? "))
ganancia_2 = ganancia_venta(precio_2, costo_2)
```

Cada venta llama a la misma función, pero cada resultado se guarda en su propia variable — ninguna reemplaza a la otra.

Paso 4 — Sumamos las dos ganancias ya guardadas.

```
ganancia_total = ganancia_1 + ganancia_2
print("Ganancia total:", ganancia_total)
```

Paso 5 — Pedimos la meta y comparamos el total con ella.

```
meta_recaudacion = int(input("¿Cuál es la meta de recaudación? "))
if ganancia_total >= meta_recaudacion:
    print("¡Meta alcanzada!")
else:
    print("Todavía no se alcanza la meta.")
```

Programa completo, armado:

```
def ganancia_venta(precio_venta, costo_producto):
    return precio_venta - costo_producto

precio_1 = int(input("¿Precio de venta del primer producto? "))
costo_1 = int(input("¿Costo del primer producto? "))
ganancia_1 = ganancia_venta(precio_1, costo_1)

precio_2 = int(input("¿Precio de venta del segundo producto? "))
costo_2 = int(input("¿Costo del segundo producto? "))
ganancia_2 = ganancia_venta(precio_2, costo_2)

ganancia_total = ganancia_1 + ganancia_2
print("Ganancia total:", ganancia_total)

meta_recaudacion = int(input("¿Cuál es la meta de recaudación? "))
if ganancia_total >= meta_recaudacion:
    print("¡Meta alcanzada!")
else:
    print("Todavía no se alcanza la meta.")
```

Ejemplo 1	Ejemplo 2
Entrada: 1200 700 900 400 800	Entrada: 1200 700 900 400 1500
Salida: Ganancia total: 1000 ¡Meta alcanzada!	Salida: Ganancia total: 1000 Todavía no se alcanza la meta.

Errores típicos a evitar

Error	Qué ocurre	Cómo corregirlo
Llamar la función directo dentro de un <code>print()</code> , sin guardarla en una variable	El resultado se muestra una vez, pero no queda disponible para usarlo en un paso posterior	Guardar siempre el llamado en una variable, aunque el siguiente paso sea solo imprimirlo
Asumir que una función sin <code>return</code> "de todas formas entrega algo" porque usa <code>print()</code> internamente	La variable que la recibe queda en <code>None</code> , y cualquier cálculo posterior con ella falla o da un resultado sin sentido	Revisar si la función tiene <code>return</code> ; si no lo tiene, agregarlo antes de intentar usar su resultado

Práctica Independiente

Revisa en tu celular: cuando termines cada ejercicio, revisa tu respuesta en el celular — la solución completa está en el cuaderno Solucionario subido a Classroom.

Ejercicio 1 — Biblioteca del liceo. La biblioteca del liceo cobra una multa por cada día de atraso en la devolución de un libro. A quienes se atrasan por primera vez, la encargada les hace un descuento fijo de \$500 sobre la multa calculada, para no desanimarlos de seguir pidiendo libros — nunca menos de \$0.

El programa debe:

- Definir una función que reciba los días de atraso y el valor de la multa por día, y devuelva el monto total de la multa con `return`.
- Pedir los días de atraso y el valor por día, llamar la función y guardar el resultado en una variable.
- Restarle a ese resultado guardado el descuento de \$500 (sin bajar de \$0) y mostrar el monto final a pagar.

Ejemplo 1	Ejemplo 2
Entrada: 5 200	Entrada: 1 200
Salida: Debe pagar de multa: 500	Salida: Debe pagar de multa: 0

Escribe tu programa aquí:

Ejercicio 2 — Radio escolar. La radio escolar arma bloques de canciones para el recreo. Cada bloque dura una cantidad de minutos según cuántas canciones tenga y el promedio de duración de cada una. Quieren saber cuántos bloques iguales necesitan para llegar a la meta de minutos de transmisión del día.

El programa debe:

- Definir una función que reciba la cantidad de canciones y la duración promedio por canción, y devuelva la duración total del bloque con **return**.
- Pedir esos dos datos, llamar la función y guardar el resultado.
- Pedir la meta de minutos del día y usar el resultado guardado para calcular cuántos bloques iguales se necesitan para alcanzarla, mostrando ese número.

Ejemplo 1	Ejemplo 2
Entrada:	Entrada:
4	5
3	2
60	40
Salida:	Salida:
Bloques necesarios: 5	Bloques necesarios: 4

Escribe tu programa aquí:

Ejercicio 3 — Torneo interno del club de ajedrez. El club de ajedrez del liceo organiza un torneo interno y quiere calcular el puntaje final de cada jugador: puntos por partidas ganadas, más un bono si terminó invicto (sin perder ninguna).

El programa debe:

- Definir una función que reciba las partidas ganadas y devuelva el puntaje base con **return** (cada partida ganada vale 10 puntos).
- Pedir las partidas ganadas y las partidas perdidas, llamar la función y guardar el puntaje base.
- Si las partidas perdidas son 0, sumarle al puntaje guardado un bono de 15 puntos por terminar invicto; si no, dejarlo igual.
- Mostrar el puntaje final del jugador.

Ejemplo 1	Ejemplo 2
Entrada: 4 0	Entrada: 3 1
Salida: Puntaje final: 55	Salida: Puntaje final: 30

Escribe tu programa aquí:

Ejercicio 4 — Desafío: Feria de Emprendimiento (todas las ventas del día). *(opcional)*

El mismo puesto de la Feria de Emprendimiento del ejemplo guiado ahora quiere repetir el cálculo de ganancia para todas las ventas del día, sin cerrar el programa entre una venta y otra, y saber la ganancia acumulada al final.

El programa debe:

- Reutilizar la misma función de ganancia de una venta (precio de venta menos costo del producto).
- Repetir el cálculo para distintas ventas hasta que se escriba "listo" en vez de un precio.
- Ir acumulando la ganancia de cada venta guardada en una variable de total, y mostrarla al terminar.

El programa imprime:

```
¿Precio de venta? (o "listo" para terminar) 1200
¿Costo del producto? 700
Ganancia de esa venta: 500
¿Precio de venta? (o "listo" para terminar) 900
¿Costo del producto? 400
Ganancia de esa venta: 500
```

```
¿Precio de venta? (o "listo" para terminar) listo  
Ganancia acumulada del día: 1000
```

Escribe tu programa aquí:

*Vas bien. Guardar siempre el resultado antes de usarlo es un paso chico que evita errores grandes
— ese hábito te va a servir en programas mucho más largos que este.*